Gerador de música através de uma RNN

Laura Barreto - uc2023214375@student.uc.pt

Beatriz Martins - uc2023210538@student.uc.pt

Catarina Marques - uc2023215471@student.uc.pt

Departamento de Engenharia Informática da Universidade de Coimbra, DEI

Abstrato

A música sempre esteve presente nas nossas vidas. Fosse numa cantiga de embalar, numa atuação da escola, ou até num café ou nos conteúdos digitais a que todos temos acesso nos dias de hoje. Atualmente, este elemento tão importante tem vindo a ser criado de outras formas mais avançadas e tecnológicas. Estas inteligências que criam arte, têm sido imensamente uma fonte de material para músicos profissionais e amadores ou até só um utilizador que tenha a mera intenção de se divertir e entreter com estas ferramentas de áudio. O nosso trabalho pretende analisar o funcionamento dessas ferramentas que geram áudio, através de um modelo que, treinado, cria a sequência de notas prevista por ele, dado um determinado áudio inicial.

1. Introdução

Geradores de conteúdo são muito populares atualmente por serem uma grande ajuda em diferentes áreas. Ferramentas que consigam criar imagens, áudio, texto e outros elementos são extremamente importantes, especialmente para estudantes e trabalhadores. Neste trabalho, será abordado o funcionamento de uma RNN que permite gerar música, que pode ter aproveitamento tanto no setor musical como no marketing, na publicidade e no entretenimento, entre outros setores.

2. Objetivos do trabalho

Este trabalho pretende gerar notas musicais usando uma RNN. O objetivo é treinar o modelo usando uma coleção de ficheiros MIDI do dataset Maestro. Desta forma, depois de treinado, o modelo será capaz de completar uma sequência de notas, prevendo quais serão as notas seguintes. Este projeto transmite ainda conhecimento acerca do funcionamento de geradores de conteúdo, dando assim a conhecer, a quem a ele tenha acesso, os diferentes aspetos relacionados a este tópico.

3. Materiais utilizados

Para este trabalho, utilizámos diversos materiais entre os quais: o código usado para criar o modelo que se encontra no Google Colab (link na bibliografia) e que foi posteriormente alterado para teste. A pasta zip do projeto contém ainda o áudio inicial e um outro ficheiro MIDI com o output esperado para esse áudio. Para além disso, decidimos ser um pouco mais criativas e juntar ao nosso trabalho uns áudios com instrumentos diferentes para mostrar a variedade de opções que tínhamos ao alterar o código, como será mencionado mais à frente.

4. O que é uma RNN?

As RNN's ou redes neuronais recorrentes são um tipo de rede neuronal treinada através de dados sequenciais como vídeos, áudios, séries temporais e texto, entre outros, onde a ordem é essencial. Estas redes possuem a capacidade de armazenar informações de forma cíclica, funcionando como uma espécie de memória e compreendendo, desta forma, operações anteriores e atuais e as possíveis relações entre elas. Estas redes são muito utilizadas para diferentes finalidades, como por exemplo: reconhecimento de fala, tradução automática, análise de emoções e sentimentos, geração de conteúdos, biologia, medicina, deteção de anomalias, jogos, entre outros.

4. Análise do modelo

4.1. Setup

Nesta primeira etapa, instalamos as bibliotecas que serão necessárias usar no código desenvolvido, entre as quais o *pyfluidsynth* e o *pretty_midi*. Depois importamos as que iremos utilizar: *collections*, *datetime*, *fluidsynth*, *glob*, *numpy*, *pathlib*, *pandas*, *pretty_midi*, *seaborn*, *tensorflow*, *display*, *matplotlib* e *optional*. *Collections* permite trabalhar com coleções de dados de maneira mais eficiente. *Fluidsynth* importa a biblioteca *pyFluidSynth*, um sintetizador de áudio em tempo real que é capaz de gerar som a partir de arquivos SF2 (SoundFont). *Glob* permite encontrar todos os caminhos de arquivos que correspondem a um padrão específico num diretório utilizando wildcards (caracteres ou padrões utilizados em programação). *Pathlib* constitui uma forma orientada a objetos de trabalhar com caminhos de diretórios e arquivos, usando objetos que encapsulam a lógica da manipulação de caminhos em vez de strings. *Pretty_midi* fornece maneira de manipular os arquivos *MIDI*. *Seaborn* é baseado em matplotlib, permitindo a visualização de gráficos, sendo ideal, em conjunto com pandas, para explorar e apresentar dados de forma mais eficiente e lidando com datasets amplos. *Tensorflow* é uma plataforma de código aberto que é usada maioritariamente para machine learning, no treino de modelos de deep learning. *Display* é uma biblioteca importada de *IPython* e funciona de forma a exibir objetos como imagens, gráficos e outros tipos de mídia mais facilmente. *Optional* importada de *typing* é usada para indicar que uma variável pode ser ou não de um tipo específico. Por último, ainda neste primeiro tópico definimos a semente aleatória (seed = 42), que será utilizada para gerar números aleatórios, levando a que sempre que o código seja executado com este valor de semente, os números gerados sejam os mesmos. Atribuímos a semente aleatória para o *TensorFlow* e para o *NumPy* e, posteriormente, definimos a taxa da amostragem (SAMPLING_RATE) para 16000 Hertz. A taxa de amostragem refere-se ao número de amostras de áudio por segundo. Foi definida para este valor porque é muito comum estar associado às tarefas de reconhecimento de fala. A escolha do parâmetro SAMPLING_RATE influencia em muito a qualidade do som e a extensão de frequências de um áudio originado. Por exemplo, quanto maior este parâmetro, maior a qualidade do som e a extensão das frequências e vice-versa.

4.2. Download the Maestro Dataset

Neste tópico, passamos a definir o caminho para o diretório dos dados através de *pathlib*, verificando se o diretório em questão existe, evitando que o download seja feito todas as vezes que o código é executado. Por fim, baixamos e extraímos o arquivo através do *TensorFlow*, caso esse diretório não exista. Através de *glob* sabemos quantos ficheiros *MIDI* o dataset contém e é impresso "Number of files: 1282".

4.3. Process a *MIDI* file

Passando ao processamento de um ficheiro *MIDI*, primeiramente é criado um objeto da biblioteca *Pretty Midi*, através de um elemento de uma lista que contém os ficheiros deste tipo anteriormente mencionados (filenames). Desta forma, é possível ter acesso a informações como: os instrumentos, as notas e o ritmo, que podem ser manipulados. Na função *display_audio* criada no código, podemos ajustar a duração dos áudios que serão completados pelo modelo, posteriormente. Nesta etapa, através da função *display_audio*, anteriormente mencionada, é possível criar e exibir o player de áudio, permitindo-nos reproduzir o tipo de *MIDI's* usados no nosso modelo. É possível, para além de tudo isto, visualizar a quantidade de instrumentos utilizados e ainda o nome deles. Podemos, através de um código alternativo abaixo apresentado, decidir o instrumento que queremos que seja usado no áudio, apenas com o intuito de explorar os materiais e usar como teste.

```
def display_audio(pm: pretty_midi.PrettyMIDI, seconds= 30):
  waveform = pm.fluidsynth(fs=_SAMPLING_RATE)
  # Take a sample of the generated waveform to mitigate kernel resets
  waveform_short = waveform[:seconds*_SAMPLING_RATE]
  return display.Audio(waveform_short, rate=_SAMPLING_RATE)

pm.instruments [0].program = pretty_midi.instrument_name_to_program ("nome_do_instrumento")
instrument = pm.instruments [0]
instrument_name = pretty_midi.program_to_instrument_name(instrument.program)
```

Figura 1 - Código alterado

4.4. Extract notes

Cada nota é representada por três variáveis: pitch, step e duration. O pitch é o tom da nota. O step é o tempo decorrido desde o fim da nota anterior até ao início da nota seguinte. A duração é o tempo em segundos da nota e é a diferença entre o fim e o início desta. Por ser mais fácil interpretar nomes de notas em vez de tons (valores numéricos), usámos uma função para converter o valor do tom (pitch) no nome da nota correspondente (exemplo: pitch = 81 corresponde à nota 'A55'). Foi escolhida uma música (presente na pasta zip do projeto e tem o nome de "input.midi") e procedemos a várias análises. Para visualizar esta peça musical, fizemos dois gráficos do pitch. O primeiro contém as primeiras 100 notas e o segundo contém todas as notas do DataFrame.

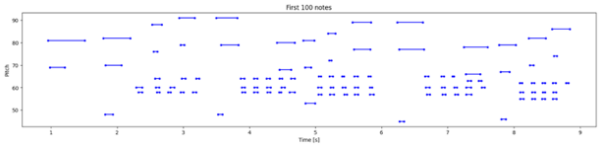

Figura 2 – Primeiras 100 notas

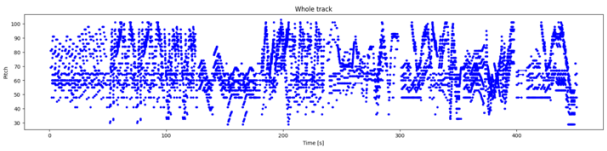

Figura 3 – Todas as notas do DataFrame

Pelo gráfico da figura 2, podemos retirar que, ao longo do tempo, os valores que lá constam são os valores dos vários tons das notas em cada

segundo. Para além disso, também é possível analisar que existe uma certa sequência neste gráfico. Para tons abaixo de 65 é bem visível esta repetição do intervalo de 2 a 3.2 segundos. Já para valores acima de 65, conseguimos perceber que os tons estão consecutivamente a subir, ou seja, há uma repetição do intervalo de 2 a 4 segundos. Observando a figura 3, com todas as notas, concluímos que esta sequência também se verifica de uma forma geral ao longo da música, principalmente a sequência dos tons abaixo de 65 nos primeiros 200 segundos. Fomos visualizar, também, a distribuição de cada variável.

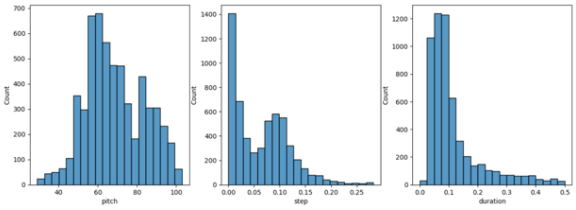

Figura 4 - Distribuição do tom, intervalo e duração

Pelo gráfico acima relativo ao tom, conseguimos observar que a maioria das notas tem um pitch entre 57 e 74 (aproximadamente), o que significa que a música tem maioritariamente um registo "confortável", ou seja, não contém, em tanta quantidade, tons nem tão altos nem tão baixos (registo médio). No entanto, como podemos ver também existem notas mais agudas e graves que não dominam a peça. Por isso, a faixa apresenta uma sonoridade equilibrada e acessível, e a presença de notas mais agudas e graves indica variação e contraste, que não se restringe apenas a um espetro de tonalidades, o que enriquece a música. No gráfico relativo ao step, podemos visualizar que o intervalo de tempo entre notas é mínimo nunca durando mais do que 0.3 segundos e normalmente dura sempre menos que 0.20 segundos. No gráfico da duração, conseguimos retirar que as notas são quase sempre de rápida duração (por norma 0.1 segundos) mas também existem notas mais longas (até 0.5 segundos). Ou seja, não se verificam notas com durações muito discrepantes das restantes, o que traz equilíbrio e suavidade ao espetro musical.

4.5. Create a *MIDI* file

Nesta etapa, temos uma função que cria um ficheiro *MIDI* onde são guardadas as notas de uma sequência. Este poderá ser útil para guardar as notas geradas como output no tópico "Generate notes", permitindo que o player para reproduzir o ficheiro *MIDI* seja criado normalmente (através da função *display_audio*).

4.6. Create the training dataset

Para criarmos um dataset de treino, passámos por um processo de extração das notas de ficheiros *MIDI* já criados através do procedimento do tópico anterior. Inicialmente, usamos um número mais reduzido desses ficheiros e fomos aumentando o seu número para obter datasets de treino mais amplos e que nos prometem melhores resultados de output. De seguida, treinámos o modelo em batches (subconjuntos de dados processados ao mesmo tempo) constituídos pelas sequências de nota anteriormente extraídas. Nesta etapa, podemos alterar o tamanho das sequências musicais e experimentar treinar os dados para verificar qual o que nos permite obter resultados mais favoráveis.

4.7. Create and train the model

O modelo deverá prever o tom, o intervalo de espera desde o fim da

nota anterior e o início da nota prevista e a duração da nota, para todas as notas previstas posteriores à nossa sequência inicial. De forma a penalizar o modelo, caso este esteja a prever valores errados nestes últimos dois parâmetros, dispomos de uma função de perda personalizada baseada no erro quadrático médio entre o valor real (y_true) e o valor previsto (y_pred), para, desta forma, tratarmos de erros negativos como sendo todos positivos e para haver uma maior penalização. Para além disto, esta função também incentiva o modelo a produzir valores não negativos, penalizando o mesmo quando isso acontecer, daí a função ser denominada por *mse_with_positive_pressure()* e que está representada na figura 7. Portanto esta função retorna o valor da perda de um parâmetro que o modelo tentará minimizar durante o treinamento.

```
def mse_with_positive_pressure(y_true: tf.Tensor, y_pred: tf.Tensor):
  mse = (y_true - y_pred) ** 2
  positive_pressure = 10 * tf.maximum(-y_pred, 0.0)
  return tf.reduce_mean(mse + positive_pressure)
```

Figura 5 - Função mse_with_positive_pressure

Já para calcular o valor de perda do tom, usamos *tf.keras.losses.SparseCategoricalCrossentropy(from_logits=true)*, que calcula a entropia cruzada entre a distribuição das probabilidades previstas pelo modelo e a distribuição de probabilidades reais. Ou seja, primeiramente aplicamos a função softmax aos logits (que expressam o quanto o modelo "acredita" em cada classe ou nota possível), de forma a transformar estes valores em probabilidades (valores entre 0 e 1), como está representado na figura 6. Portanto, se a nota correta estiver associada a uma grande probabilidade, então o nosso modelo está a prevê-la corretamente e, por conseguinte, terá um valor de perda muito baixo. Caso contrário, será mais elevado consoante a sua probabilidade.

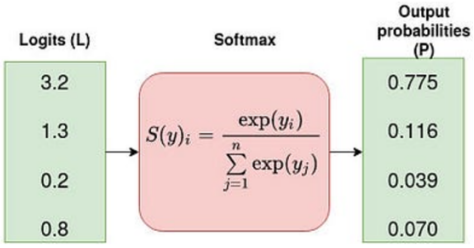

Figura 6 - Função Softmax

No sentido de minimizar o valor de perda, utilizámos um algoritmo de otimização estocástico denominado Adam (Adaptive Moment Estimation) com uma taxa de aprendizagem de 0.005 (que controla o tamanho dos passos, no sentido contrário do gradiente, que o algoritmo dá ao ajustar os parâmetros do modelo). Este método é uma combinação do algoritmo Momento e AdaGrad. O primeiro ajuda a "acelerar" o processo de aprendizagem e a ideia é dar mais importância às direções que já foram consistentes ao longo do tempo, evitando oscilações e melhorando a convergência. Por sua vez, o segundo, também conhecido por adaptative gradient ajusta a taxa de aprendizagem para cada parâmetro individualmente, baseando-se na frequência de atualizações anteriores para, desta forma, compensar o facto de algumas variáveis raramente serem atualizadas.

Descida de gradiente estocástica - Adam
Parâmetros: β_1 e β_2 em $(0,1)$ e taxa de aprendizagem λ.

- Inicializar x_0 e $m_0, v_0 = 0$.
- Iterar para $k = 0, 1, 2, ...$
  - Escolher $i \in \{1, 2, ..., N\}$ e tomar $g = \nabla f_i(x_k)$;
  - Fazer $m_{k+1} = \beta_1 m_k + (1 - \beta_1) g$;
  - Fazer $v_{k+1} = \beta_2 v_k + (1 - \beta_2) g \odot g$;
  - Calcular $\hat{m} = \frac{m_{k+1}}{1-\beta_1^{k+1}}$ e $\hat{v} = \frac{v_{k+1}}{1-\beta_2^{k+1}}$
  - Para $j = 1, ..., n$, atualizar

$$[x_{k+1}]_j = [x_k]_j - \frac{\lambda}{\sqrt{\hat{v}_j} + 10^{-8}} \hat{m}_j$$

Figura 7 – Pseudocódigo do algoritmo Adam

De seguida, através do comando *model.compile()*, o modelo é compilado com as funções de perda e o otimizador definidos como parâmetros. Posteriormente, podemos visualizar o resumo da arquitetura do modelo usando *model.summary ()*.

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 25, 3)	0	-
lstm (LSTM)	(None, 128)	67,584	input_layer[0][0]
duration (Dense)	(None, 1)	129	lstm[0][0]
pitch (Dense)	(None, 128)	16,512	lstm[0][0]
step (Dense)	(None, 1)	129	lstm[0][0]

Total params: 84,354 (329.51 KB)
Trainable params: 84,354 (329.51 KB)
Non-trainable params: 0 (0.00 B)

Figura 8 – Resumo do modelo

Usando a função *model.evaluate (train_ds,return_dict=true),* concluímos, da figura 9, que o valor de perda do tom é muito mais elevado comparativamente ao da duração e do step. Isto sugere que o modelo tem mais dificuldades em aprender a prever o tom musical corretamente. De forma que o modelo não sinta tanto este impacto de perda do tom durante o treino,
usámos *model.compile(),* descrito na figura 10, e atribuiremos uma importância muito pouco significativa de 0.05 ao tom, e à duração e ao step o valor de 1. Assim, já podemos verificar, na figura 11, que a perda total já não está a ser dominada pelo tom e, consequentemente, o modelo pode aprender, de uma forma mais equilibrada, a prever os valores tanto do tom como da duração e do step.

```
'duration_loss': 0.20354239642620087,
'loss': 5.888579368591309,
'pitch_loss': 4.836723804473877,
'step_loss': 0.8483161926269531}
```

Figura 9 – Valores das diferentes perdas

```
model.compile(
    loss=loss,
    loss_weights={
        'pitch': 0.05,
        'step': 1.0,
        'duration':1.0,
    },
    optimizer=optimizer,
)
```

Figura 10 – Ajustar a importância do tom

```
'duration_loss': 0.20354239642620087,
'loss': 1.2936949729919434,
'pitch_loss': 0.2418360859155655,
'step_loss': 0.8483161926269531}
```

Figura 11 – Resultados nos parâmetros após as alterações anteriores

CallBacks são funções que permitem monitorizar o progresso de treino e executar ações específicas em determinados momentos. Os dois *Callbacks* presentes na figura 12 são *ModelCheckpoint* e *EarlyStopping*. A primeira é útil para guardar os pesos do modelo de cada época de treino e a variável "filepath" indica o caminho onde os guardar. A segunda permite interromper o treino automaticamente caso não se verifiquem melhorias no valor da perda total (especificada na variável "monitor") após cinco épocas consecutivas (número definido na variável "patience") e reproduz uma mensagem informativa sempre que o treino for interrompido devido à linha "verbose=1;".

```
callbacks = [
    tf.keras.callbacks.ModelCheckpoint(
        filepath='./training_checkpoints/ckpt_{epoch}.weights.h5',
        save_weights_only=True),
    tf.keras.callbacks.EarlyStopping(
        monitor='loss',
        patience=5,
        verbose=1,
        restore_best_weights=True),
]
```

Figura 12 – Callbacks

De seguida usamos *%%time* para medir o tempo de treinamento nas 50 épocas com dados de treino e os callbacks definidos anteriormente, como é possível observar na figura 13.

```
%%time
epochs = 50

history = model.fit(
    train_ds,
    epochs=epochs,
    callbacks=callbacks,
)
```

Figura 13 - Épocas de treino

```
Epoch 1/50
395/395 ━━━━━━━━━━━━━━━━━━━━ 42s 93ms/step - duration_loss: 0.1253 - loss: 0.3947 - pitch_loss: 0.2198 - step_loss: 0.0497
Epoch 2/50
395/395 ━━━━━━━━━━━━━━━━━━━━ 23s 48ms/step - duration_loss: 0.1062 - loss: 0.3517 - pitch_loss: 0.2056 - step_loss: 0.0399
Epoch 3/50
395/395 ━━━━━━━━━━━━━━━━━━━━ 23s 54ms/step - duration_loss: 0.1043 - loss: 0.3435 - pitch_loss: 0.2001 - step_loss: 0.0391
Epoch 4/50
395/395 ━━━━━━━━━━━━━━━━━━━━ 19s 48ms/step - duration_loss: 0.1031 - loss: 0.3390 - pitch_loss: 0.1969 - step_loss: 0.0391
...
395/395 ━━━━━━━━━━━━━━━━━━━━ 20s 50ms/step - duration_loss: 0.0407 - loss: 0.2431 - pitch_loss: 0.1761 - step_loss: 0.0262
Epoch 49/50
395/395 ━━━━━━━━━━━━━━━━━━━━ 21s 54ms/step - duration_loss: 0.0391 - loss: 0.2410 - pitch_loss: 0.1761 - step_loss: 0.0258
Epoch 50/50
395/395 ━━━━━━━━━━━━━━━━━━━━ 20s 49ms/step - duration_loss: 0.0398 - loss: 0.2411 - pitch_loss: 0.1754 - step_loss: 0.0258
Restoring model weights from the end of the best epoch: 50.
CPU times: user 22min 5s, sys: 1min 12s, total: 23min 17s
Wall time: 21min 30s
```

Figura 14 - Output das épocas de treino

Podemos observar, na figura 14, que o valor total de perda vai diminuindo ao longo das épocas de treino, tendo o valor de 0.3947 na primeira época e na última época tendo o valor de 0.2411, ou seja, ao longo do treinamento, conseguimos minimizar o valor da perda de total e, desta forma, o modelo está a aprender a prever valores cada vez mais corretos. Também podemos observar que este treino demorou 23min e 17s pois usámos 7 files no treinamento. Analisando o gráfico da figura 15, podemos visualizar de uma forma mais intuitiva, que houve uma minimização no valor de perda, ao longo das épocas, com alguns picos mínimos em que o valor de perda piorou (como é o caso da época 25), mas, de uma forma geral, podemos concluir que a etapa de treinamento foi realizada com sucesso.

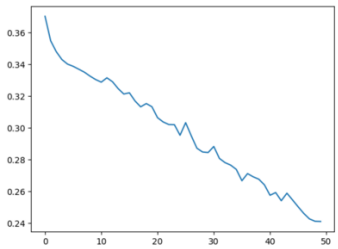

Figura 15 – Gráfico da evolução do valor da perda total ao longo das épocas devido ao valor da paciência de valor *5*

Também pudemos verificar que numa das vezes que treinámos o modelo, houve uma interrupção devido à função Earlystopping, por não se verificarem melhorias significativas a partir da época 8, como é observável nos gráficos seguintes, tendo sido interrompido o treinamento na época 13, 5 épocas a seguir à 8. Por fim, retorna a época com melhor valor de perda, como já tínhamos visto anteriormente, sendo este a época 8.

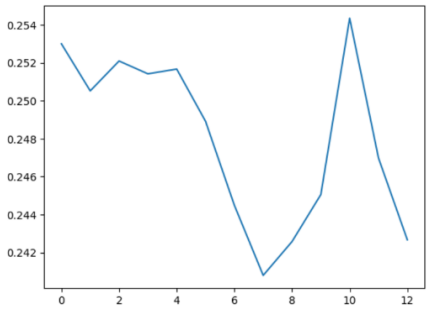

Figura 16 – Gráfico das perdas em um treino no caso de EarlyStopping

...

```
Epoch 8/50
149/149 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - duration_loss: 0.0497 - loss: 0.2390 - pitch_loss: 0.1758 - step_loss: 0.0135
Epoch 9/50
149/149 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - duration_loss: 0.0509 - loss: 0.2408 - pitch_loss: 0.1760 - step_loss: 0.0138
Epoch 10/50
149/149 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - duration_loss: 0.0496 - loss: 0.2378 - pitch_loss: 0.1752 - step_loss: 0.0129
Epoch 11/50
149/149 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - duration_loss: 0.0615 - loss: 0.2523 - pitch_loss: 0.1760 - step_loss: 0.0148
Epoch 12/50
149/149 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - duration_loss: 0.0610 - loss: 0.2525 - pitch_loss: 0.1751 - step_loss: 0.0164
Epoch 13/50
149/149 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - duration_loss: 0.0575 - loss: 0.2465 - pitch_loss: 0.1747 - step_loss: 0.0142
Epoch 13: early stopping
Restoring model weights from the end of the best epoch: 8.
CPU times: user 12.4 s, sys: 659 ms, total: 13.1 s
Wall time: 14.5 s
```

Figura 17 – Valores das perdas nas épocas de um treino no caso de EarlyStopping

4.8. Generate notes

Como seria de esperar, temos a função *predict_next_notes()* que devolve um tuplo com o tom, o intervalo e a duração da nota prevista, e recebe como parâmetros a sequência de notas, o modelo e a temperatura. A temperatura controla o grau de aleatoriedade na geração dos parâmetros da próxima nota, portanto para temperaturas mais baixas são escolhidas as opções com a maior probabilidade, o que incentiva a um modelo mais determinístico e

conservador. Caso contrário teremos saídas mais aleatórias, o que potencializa alguma criatividade à sequência musical. No entanto, ao permitir uma maior criatividade, os áudios acabam por, na maioria das vezes, ser um pouco dissonantes e até criar uma sequência que nada tem a ver com a inicial. Desta forma, decidimos utilizar uma temperatura de 0.5, com a qual obtivemos um melhor resultado. Usámos o modelo para prever os três parâmetros com *model.predict(inputs)* e é necessário dividir os logits do tom pela temperatura, uma vez que esta ajusta a nitidez e suavidade da distribuição das probabilidades. Posteriormente, usámos a função *tf.random.categorical()* e atribuímos os parâmetros pitch_logits e num_samples = 1). Esta retorna um pitch escolhido com base nas probabilidades. Assim, conseguimos prever a quantidade de notas que queremos, atribuindo esse valor à variável num_preditions. Através de um ciclo for de 1 a num_preditions, conseguimos determinar os três parâmetros de cada nota, sendo que, em cada iteração, só consideramos um número fixo de notas anteriores como parâmetro da função *predict_next_notes(),* o que exige o cuidado de eliminar a nota mais antiga no final de cada iteração, de forma a já não ser considerada na previsão da nota seguinte. Para além disto, também é importante referir que as notas geradas são guardadas num vetor (que posteriormente passámos para um dataFrame) juntamente com o seu tempo de início e de fim para termos uma sequência temporal das notas. Inclusive as 10 primeiras notas podem ser observadas na figura 18.

	pitch	step	duration	start	end
0	65	0.071948	0.115763	0.071948	0.187711
1	89	0.090606	0.097752	0.162554	0.260306
2	96	0.142450	0.275034	0.305004	0.580038
3	96	0.154635	0.382001	0.459639	0.841640
4	89	0.155621	0.431604	0.615260	1.046864
5	96	0.157056	0.461561	0.772316	1.233878
6	93	0.155406	0.449065	0.927723	1.376787
7	89	0.155783	0.457429	1.083505	1.540935
8	91	0.158930	0.478184	1.242436	1.720619
9	96	0.170371	0.522571	1.412807	1.935378

Figura 18 – Primeiras 10 notas

Usando a função *notes_to_midi()*, obtivemos o áudio das notas previstas pelo modelo (áudio que se encontra na pasta zip com o nome de "output.mid") que, comparando com as notas iniciais, parecem seguir de uma forma razoável os tons das notas do ficheiro de input, no entanto existem notas mais agudas.

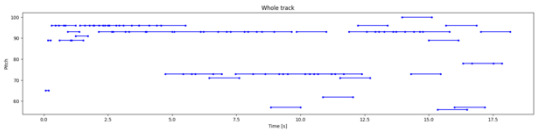

Figura 19 – Gráfico dos tons previstos no output

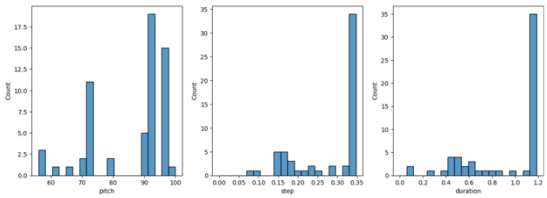

Figura 20 – Distribuição do tom, intervalo e duração das notas geradas no output

Verificando os valores de pitch, podemos evidenciar uma divisão em classes e que existiu um aumento no tom das notas. Já no step e na duração, podemos comprovar que o modelo foi muito determinístico e conservador, porque a maioria dos intervalos entre notas e a duração são 0.35 e 1.15 segundos respetivamente.

5. Discussão e resultados

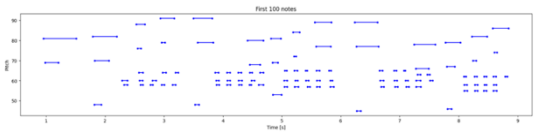

Figura 21 – Gráfico inicial dos tons do input

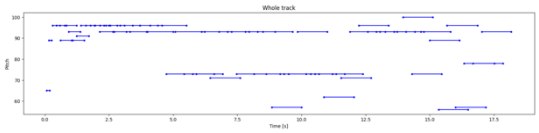

Figura 22 – Gráfico final dos tons do output

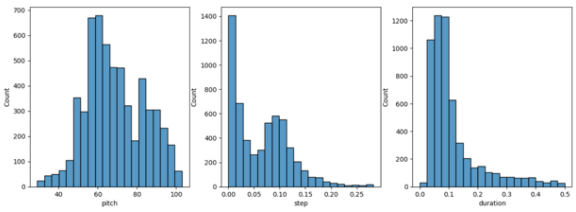

Figura 23 – Distribuições do tom, intervalo e duração do input

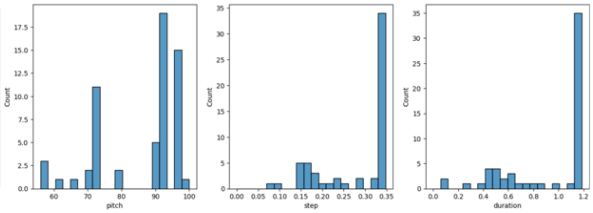

Figura 24 – Distribuições do tom, intervalo e do output

Após todos os métodos e procedimentos realizados anteriormente, podemos tirar algumas conclusões e resultados. Comparando os primeiros dois gráficos do input e output, podemos concluir que estão ligeiramente diferentes, no entanto o modelo aparenta ter escolhido duas classes de tons que mais se repetem no input, por serem mais prováveis. Uma delas tem um tom de 56 a 72 e a outra tem um tom de 80 a 100, aproximadamente. No tom das notas geradas, parece então, ser bem visível a presença destas duas classes. As conclusões retiradas dos dois últimos gráficos estão relacionadas com o intervalo e a duração. Ao observá-los, podemos entender de imediato a grande diferença entre os dois, nesses parâmetros. Analisando, retiramos que o intervalo das notas geradas teve valores muito parecidos aos obtidos no input, diferindo apenas em algumas décimas. Já na duração, a maior parte dos valores passaram a ser o dobro do valor mais alto dos valores iniciais no input e por mais que treinássemos o modelo alterando diferentes parâmetros (temperatura, número de ficheiros) não conseguimos combater esta adversidade, no entanto este áudio gerado foi o mais aceitável e parecido com o input.

6. Conclusão

Através deste trabalho, conseguimos gerar uma continuação de notas para uma sequência musical inicial. Experimentando diferentes valores para os vários parâmetros de treino, tivemos contacto com diversos resultados, que nos permitiram entender de uma forma mais fácil o funcionamento de um modelo e de uma RNN. Concluindo, com este projeto, esperamos que todos consigam ter um maior entendimento acerca deste assunto tão relevante, cativante e inovador.

7. Referências

[1] Faria, B., Perdigão, D., Brás, J., Macedo, L.: The Joint Role of Batch Size and Query Strategy in Active Learning-based Prediction - A case study in the Heart Attack Domain, (2022), UC Student (Repositório de Tópicos de Inteligência Artificial)

[2] Pinto, C. Faria, J., Macedo, L.: An Active Learning-based Medical Diagnosis System*, (2022), UC Student (Repositório de Tópicos de Inteligência Artificial)

[3] Vieira, P., Félix, P., Macedo, L.: Open-World Active Learning with Stacking Ensemble for Self-Driving Cars, (2021), UC Student (Repositório de Tópicos de Inteligência Artificial)

[4] Chat GPT: usado para traduzir alguns trechos e analisar algumas funções por nós desconhecidas

[5] **Função softmax:** https://ichi.pro/pt/funcao-de-ativacao-softmax-como-realmente-funciona-231528050572664

[6] **Código original:** https://colab.research.google.com/github/tensorflow/docs/blob/master/site/en/tutorials/audio/music_generation.ipynb?hl=pt#scrollTo=kahm6Z8v_TqC

[7] **Código alterado:** https://colab.research.google.com/drive/1teN77f2jfA63jli_-ms3Vc5eCXcfH0al

[8] Falámos com o professor João Gouveia que leciona a cadeira de "Otimização Contínua" para entender melhor o funcionamento do Otimizador ADAM

[9] Apontamentos de Otimização Contínua relativos ao Otimizador ADAM